

Agate: Capability Microkernels as a Natural Substrate for AI Agents

Zhao Wei
Huawei Technologies
aznikline@gmail.com

Bin Fang
Huawei Technologies
fang.bin@huawei.com

ABSTRACT

AI agents are becoming always-on services that plan, invoke tools, collaborate, and dynamically generate code—workloads that demand fine-grained tool authorization, structured inter-agent communication, code sandboxing, decision provenance, and versioned long-lived context. We argue that **capability-based microkernels** are a natural substrate for agent workloads by structural correspondence, not analogy: their three primitives—capability-mediated authorization, IPC with capability passing, and per-agent address-space isolation—supply the enforcement core for three of six foundational desiderata considered here (D1–D3). We *implement* in **Agate**, an AArch64 capability microkernel (45 syscalls, 12 capability types, a complete VM subsystem with demand paging and CoW), three extension layers—IPC-path provenance, context-object state, and semantic deadline scheduling—addressing three additional desiderata (D4–D6) without redesigning the capability/IPC/isolation core. The kernel mechanisms backing D1–D6 are implemented and unit-tested; a bounded EL0 agent→tool→reply round-trip is closed. An autonomous end-to-end agent workload evaluation, comparative performance, multi-core correctness, and formal verification remain future work.

KEYWORDS

capability microkernels, AI agents, operating systems, isolation, IPC, provenance

1 THE AGENT–OS MISMATCH

An LLM-based agent is a long-lived process: it holds context across turns, invokes external tools (search, code execution, APIs), spawns sub-tasks, and talks to other agents. This stresses three traditional OS abstractions:

- **Coarse isolation.** A process carries ambient authority—any file its UID can reach. Linux narrows this with sec-comp, Landlock, file capabilities, and namespaces, but each is a retrofit composed in userspace; confinement is policy the caller must assemble, not the default grant model. An agent generating and executing untrusted code cannot confine that code without layering such machinery atop the kernel.
- **Unstructured communication.** Sockets carry byte streams; the OS has no notion of a *tool invocation* or a *capability grant*. Agent runtimes such as LangChain and AutoGen re-implement tool authorization and message typing in userspace, per-runtime—often duplicating grants, role checks, and sandbox setup that are then unauditable from below.

Table 1: Structural mapping: agent desiderata ↔ microkernel primitives. D1–D3 are native to the trinity; D4–D6 are the three contributed layers (Section 3).

Desideratum	Microkernel primitive	Agate feature
D1: authorize tools	Capability (endpoint + grant, recursive revoke)	12 cap types, derivation tree, subtree revocation; immutable Image caps bind startup caps to a SHA-256 ELF digest
D2: inter-agent comm.	IPC + cap passing (kernel-stamped sender)	Send/reply-recv, fastpath, anti-forge identity
D3: isolate generated code	Per-agent vspace + CoW fork	Independent page tables, demand paging, CoW fork

- **Passive state.** Files are opaque; the OS cannot observe an agent’s context or memory being mutated or forked. Traditional OSes do not natively represent tool-level, cross-runtime invocation provenance as a uniform authority-mediated abstraction, so a misbehaving tool call cannot be attributed across runtimes.

The cost is concrete: these mechanisms are re-implemented per framework, outside the kernel where they could be enforced uniformly.

2 THESIS: A STRUCTURAL MAPPING

Capability microkernels—in the tradition of seL4 [1] and L4 [2], here Agate—provide three primitives that correspond to agent needs. We make the claim precise rather than rhetorical. Enumerate the agent desiderata: **(D1)** least-privilege tool authorization with uniform, cascading revocation; **(D2)** structured, identity-bearing inter-agent communication; **(D3)** cheap, isolated sandboxes for generated code; **(D4)** decision provenance (who called which tool, with what result); **(D5)** versioned, long-lived context state; **(D6)** latency-aware scheduling, so an agent awaiting a user response can preempt background work. The microkernel trinity covers **D1–D3** natively; **D4–D6** are the three layers we contribute in Section 3, each additive in that it introduces a new capability type, audit field, or scheduler entry without redesigning the core. “Natural substrate” we use operationally: an agent-OS can be built by layering capability primitives onto a microkernel base without redesigning its capability/IPC/isolation core—not that the base is already sufficient.

Worked scenario. An agent that must run generated code: (i) the supervisor uses Agate’s agent-spawn path to create a CoW-backed child with an independent vspace and an empty root CN-ode; (ii) it injects only an attenuated code-exec endpoint capability; (iii) the child invokes the tool through IPC, which kernel-stamps the caller; (iv) revocation recursively removes that delegated capability and its descendants as one logical kernel operation. No ambient file authority, no per-runtime sandbox script.

Why this is not just “also works” against a monolithic baseline. A skeptic’s real baseline is Linux + seccomp + namespaces + Landlock + eBPF, or a restricted-token sandbox such as OpenAI Codex’s Windows sandbox [8], which strips ambient authority via restricted tokens and per-object ACLs—already fine-grained and runtime-revocable. The microkernel’s honest advantage is narrower and structural: (1) *revocation composes by construction*—destroying a capability cascades to all derived grants as a single logical kernel operation, whereas pidfd/cgroup-freezer/seccomp or ACL revocation is per-mechanism and does not transitively retract every derived access. (2) *Identity is kernel-stamped at the chokepoint*—IPC stamps the sender at the only path that can reach a capability. (3) *No ambient authority is the grant model*, not an assembled policy. Capsicum [6] and CHERI [7] narrow this gap.

Assumption and its limits. Provenance via (2) holds *only under the discipline that inter-agent calls are IPC-mediated*. The capability model already denies a process any handle it was not granted, so an agent without a shared-memory capability cannot bypass the chokepoint—but this is a design contract, not a law of nature; Section 3 returns to it.

3 THREE EXTENSION LAYERS

The microkernel base is necessary but not sufficient: D4–D6 are *not* covered by the trinity and are the three layers we contribute. Each introduces a new audit field, capability type, or scheduler entry; *none redesigns the capability/IPC/isolation core*—“additive” means the defining constraints are preserved, not that the layers are minor. The capability set a tool needs and the deadline an agent owes are not discovered by the kernel: they are supplied by a declarative tool manifest and a per-tenant policy broker. Generating tight specifications is a long-standing open problem outside this paper’s scope; the kernel’s contribution is enforcing them once supplied.

(A) IPC-path provenance (D4). The credential validator runs on the SYS_IPC_SEND path before the fastpath branch, so both fast and slow paths are kernel-stamped; the audit record carries caller, endpoint, correlation identity, and event-class-dependent digest fields, queryable via SYS_AUDIT_QUERY; the fastpath is also audited, so no successful capability-mediated tool send on this path escapes the audit point (the reply-recv and notify paths are not audited in this version). Audit-v2 records, access-controlled through the CAP_TYPE_AUDIT singleton, carry event-class-dependent SHA-256 digest fields: for IPC events, those digests bind routing metadata (type/sender/receiver/length) rather than raw payload semantics; approval-lifecycle events are hashed by a userspace trusted broker

including payload bytes.¹ This yields a *kernel-authenticated IPC invocation graph*—proving message *origin* and selected digest bindings, not execution authenticity or message semantics; it does not cover shared-memory or userspace-RPC calls, so the grant policy must withhold raw shared-memory capabilities.

(B) Context-object state (D5). We layer a *context object*—an agent’s prompt/scratchpad/episodic memory, mapped into the vspace—atop the page-set object (modeled on Zircon’s [3] VmObject). Each revision is a CoW clone (version = parent+1), shared via CAP_TYPE_CONTEXT with a stable, pointer-independent identity. A Context UAPI (SYS_CONTEXT_SNAPSHOT/INFO/MAP) snapshots, introspects, and maps state; the kernel computes an exact backing-page-identity diff (merge-joining sorted offset arrays and comparing backing physical addresses, so $\text{diff}(\text{parent}, \text{unmodified snapshot}) = 0$), not a content hash.² *Semantic* diff (what a prompt change *means*) is out of scope—it requires userspace interpretation.

(C) Semantic deadline scheduling (D6). The deadline/budget come from the userspace broker; the kernel does not infer semantics. Agate previously had EDF admission but no deadline-ordered ready queue. We closed that: admission is wired at boot, an agent task is an EDF entity (deadline = user-response deadline, budget = inference slice), and a deadline-ordered runqueue drives earliest-deadline selection with per-tick budget debit/replenish. An interactive agent can then preempt a background indexer. A system-wide utilization test ($\sum C_i/T_i \leq 1.0$, SYS_SCHED_SET_DEADLINE) bounds over-subscription. No change to vspace or IPC.

Threats to validity. The mapping rests on the IPC-mediated discipline; agents granted raw shared-memory or userspace-RPC capabilities bypass the chokepoint. The kernel mechanisms underlying D4–D6 are implemented and unit-tested (the IPC-path provenance record, context-object create/snapshot/diff, scheduler runqueue); the approval-lifecycle digest binding is computed by a userspace trusted broker, not the kernel. A bounded EL0 agent→tool→reply round-trip (SYSINFO tool) is closed, but an *autonomous* end-to-end agent reasoning workload evaluation is future work—so the “additive” claim is a tested-kernel argument, not yet a measured workload result. A performance comparison against Linux+seccomp, an adversarial evaluation of the chokepoint, and integration with a realistic agent runtime remain future work, not claims of this paper.

4 RELATED WORK

seL4 [1] provides the formally verified capability model whose recursive revocation Agate follows (Agate claims no formal verification); L4 [2] established external-pager demand paging; Zircon [3] provides the VmObject layering Agate models; Genode [4] shares no-ambient-authority but targets components, not agents. Capsicum [6] and CHERI [7] provide closely related capability-oriented mechanisms at the OS and ISA layers, respectively. OpenAI’s Codex Windows sandbox [8] strips ambient authority

¹The 160-byte `agate_audit_record_v2` has monotonic sequence and a saturating lost counter; a legacy non-cryptographic FNV-1a hash coexists for some kernel events. The audit ring is *not* cryptographically chained (no Merkle tree), so ring-level tamper-evidence is out of scope.

²Identity is surfaced through a 64-byte, ABI-stable `agate_context_info_v1` struct; the page-table walk, fault resolver, and CoW core are reused.

via restricted tokens and ACLs—single-OS-layer and userspace-assembled, without cascading revocation or a kernel-stamped IPC chokepoint. The secure-OS tradition shares our confinement and attribution goals through mandatory access control, information-flow policies, and auditing; Agate differs in emphasizing capability-mediated delegation, derivation, and transitive revocation tied to IPC identity. The AgenticOS CFP [5] names isolation, scheduling, provenance, and inter-agent communication. Agent runtimes (LangChain, AutoGen) enforce authorization in userspace; we argue those mechanisms belong in the substrate, and that A2A/MCP-facing gateways could authenticate remote peers and translate them into local capability endpoints.³

5 STATUS AND CONCLUSION

The kernel mechanisms underlying D1–D6 are implemented and unit-tested (1243 passing self-tests, 0 failing); the approval-lifecycle digest binding is a userspace trusted-broker computation. A bounded EL0 agent→tool→reply round-trip is closed; a 100× lifecycle zero-resource-delta gate and a forced service-death/peer-death isolation gate surfacing `-ECONNRESET` to a blocked client call

are both closed. The validated baseline is single-CPU (QEMU virt, cortex-a72); SMP correctness, an autonomous end-to-end agent workload, comparative performance evaluation, and formal verification remain future work. In Agate, the capability/IPC/isolation core covers D1–D3 by construction, while the three extensions address D4–D6 without re-architecting that core.

REFERENCES

- [1] G. Klein, J. Andronick, K. Elphinstone, T. Murray, T. Sewell, R. Kolanski, and G. Heiser. *Comprehensive formal verification of an OS microkernel*. *ACM Transactions on Computer Systems*, 32(1):2:1–2:70, 2014.
- [2] J. Liedtke. *On μ -kernel construction*. In *SOSP*, 1995.
- [3] Google. *Zircon Kernel: VmObject and VmMapping*. Fuchsia OS Documentation, 2024. <https://fuchsia.dev/fuchsia-src/reference/kernel/vm>.
- [4] N. Feske. *The Genode Framework*. Genode Labs, 2015. <https://genode.org>.
- [5] Y. Xia, D. Du, C. Wang, et al. *AgenticOS Workshop: Call for Papers*. *SOSP 2026*. <https://os-for-agent.github.io/>.
- [6] R. N. M. Watson, J. Anderson, B. Laurie, and K. Kennaway. *Capsicum: Practical capabilities for UNIX*. In *USENIX Security Symposium*, pages 29–46, 2010.
- [7] R. N. M. Watson, J. Woodruff, P. G. Neumann, et al. *Capability Hardware Enhanced RISC Instructions (CHERI)*. In *ASPLOS*, 2015.
- [8] OpenAI. *Building the Codex Windows Sandbox*. OpenAI, 2025. <https://openai.com/index/building-codex-windows-sandbox/>.

³Cross-node capability transport and distributed revocation are not implemented or evaluated here.